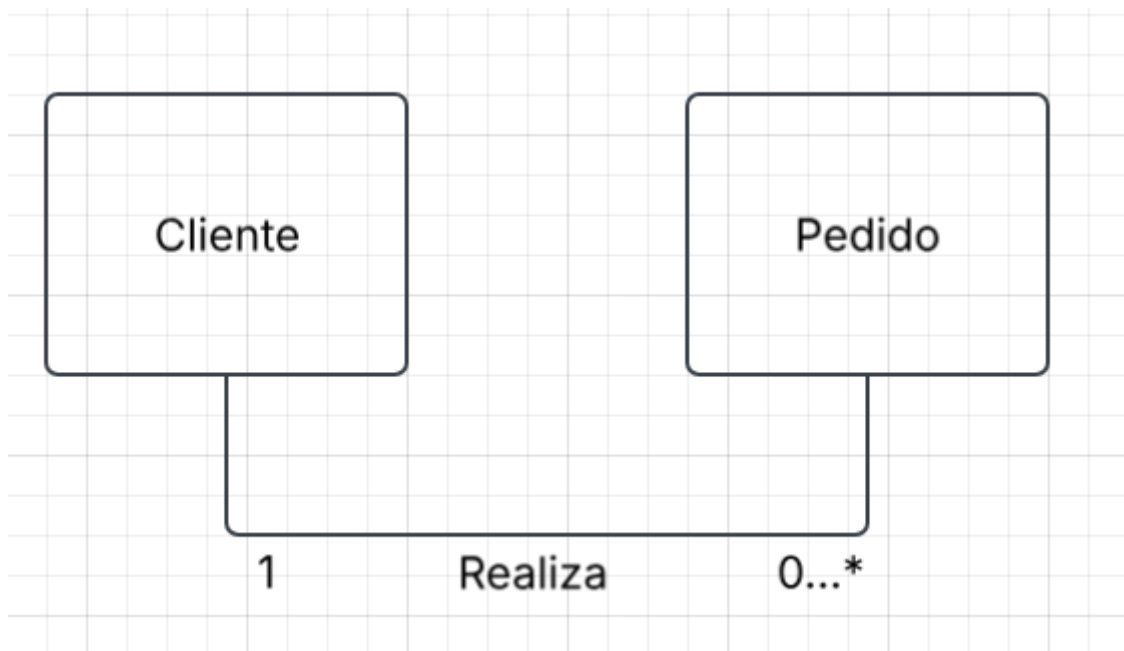


PARTE 1: Resumo Técnico – Relacionamentos entre Objetos

Na Programação Orientada a Objetos (POO), os objetos raramente trabalham sozinhos. Para modelar sistemas reais, precisamos estabelecer como eles se comunicam e se estruturam. Existem três formas principais de relacionamento:

1. Associação

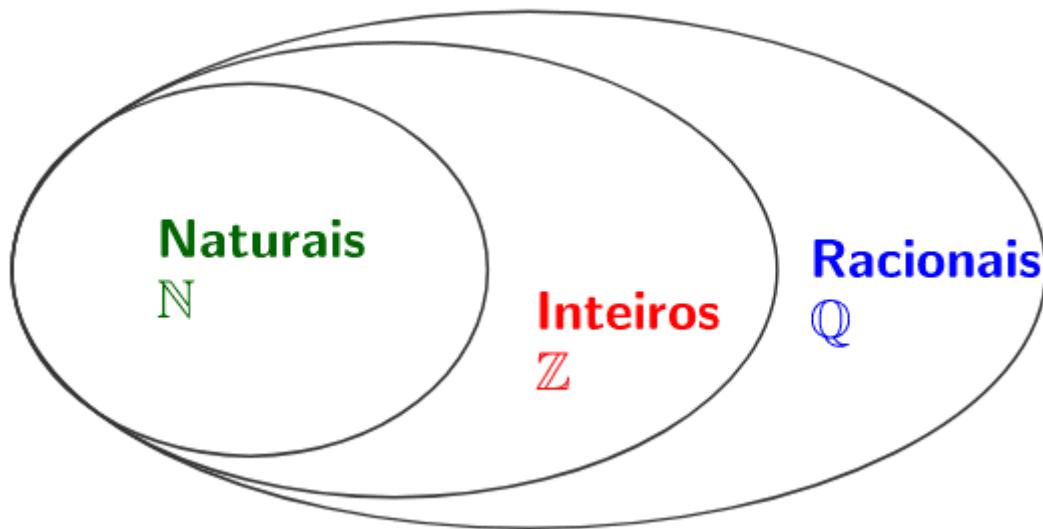
- **O que é:** É o relacionamento mais simples, onde **dois objetos possuem uma ligação (se relacionam), mas continuam sendo completamente independentes**. Eles possuem ciclos de vida separados: se um objeto deixar de existir, o outro continua existindo normalmente.
- **Exemplo conceitual:** Um **Cliente** e um **Pedido**. O pedido está associado a um cliente (sabemos quem comprou), mas se o pedido for deletado ou cancelado, o cliente continua cadastrado no sistema. Da mesma forma, se o cliente for removido, o histórico de pedidos pode continuar existindo de forma independente.



2. Agregação

- **O que é:** É um relacionamento do tipo "todo-parte", no qual **um objeto principal (o "todo") reúne ou contém outros objetos (as "partes")**, mas essas partes **ainda podem existir de forma separada** fora do todo.

- **Exemplo conceitual:** Um **Time** e seus **Jogadores**. O time é o objeto principal que agrega os jogadores. Se o time for desfeito (destruído), os jogadores não deixam de existir; eles continuam existindo de forma independente no sistema e podem ser agregados a outro time.



3. Composição

- **O que é:** É uma relação de "**todo-parte**" muito mais forte e rígida. Na composição, as partes **pertencem exclusivamente ao objeto principal** e dependem totalmente dele para existir. O ciclo de vida das partes é controlado pelo "todo": quando o objeto principal é destruído, todas as suas partes também são eliminadas.
- **Exemplo conceitual:** Um **Pedido** e seus **Itens do Pedido** (ItemPedido). Um item de pedido (ex: "2x Teclado de R\$120") não faz nenhum sentido existindo sozinho no sistema sem estar vinculado a um pedido real. Se o pedido for excluído, todos os seus itens associados são destruídos junto com ele.



☀ A Principal Diferença entre Agregação e Composição

A diferença crucial está na **independência do ciclo de vida das partes**:

- Na **Agregação**, as partes têm vida própria e **podem sobreviver** sem o objeto pai.
- Na **Composição**, as partes **dependem da existência** do objeto pai e são destruídas junto com ele.

1. Relacionamentos entre Objetos no Back-End

Em uma aplicação de servidor (Back-End), os relacionamentos de POO definem como a informação é estruturada e gerenciada:

- **Associação (Relacionamento Fraco e Independente):**

- **No Back-End:** Representa a relação entre **Cliente** e **Pedido**. O cliente existe no cadastro do sistema mesmo se não tiver feito nenhum pedido. No banco de dados, eles são tabelas ou documentos separados conectados apenas por uma chave (como clienteld). Se deletarmos o pedido, o registro do cliente continua intacto.

- **Agregação (Relacionamento "Todo-Parte" Temporário):**

No Back-End: Representa, por exemplo, um **Time** e seus **Jogadores**. Se o clube for desativado no sistema, as entidades dos jogadores não podem sumir da base de dados; eles apenas ficam "sem time" e podem ser transferidos para outro cadastro.

- **Composição (Relacionamento "Todo-Parte" Forte e Dependente):**

No Back-End: É a relação entre **Pedido** e seus **Itens do Pedido** (ItemPedido). Um item ("2 unidades do Produto X") não tem sentido de existir de forma órfã no banco de dados. No Back-End, se um pedido for excluído, o sistema executa uma **exclusão em cascata**, deletando todos os itens associados automaticamente.

3. Código JavaScript Planejado (Integrando Tudo)

Para a apresentação de 10 minutos, este código demonstra de forma clara como **Entidades de Domínio** se relacionam (composição) e como um **Serviço** coordena as operações do Back-End:

```
// =====
```

```
// 1. ENTIDADES DO DOMÍNIO (Guardam os Dados)
```

```
// =====
```

```
// Entidade que representa um item individual (Parte na Composição)
```

```
class ItemPedido {  
  constructor(nome, preco) {  
    this.nome = nome;  
    this.preco = preco;
```

```
}  
}
```

// Entidade que representa o Pedido (O "Todo" na Composição e Associação com Cliente)

```
class Pedido {  
    constructor(id, clientId) {  
        this.id = id;  
        this.clientId = clientId; // ASSOCIAÇÃO: Ligado ao cliente apenas por referência/ID  
        this.itens = []; // COMPOSIÇÃO: O Pedido é dono do ciclo de vida desta lista  
    }  
}
```

// Composição na prática: a criação de ItemPedido ocorre dentro da própria entidade pai

```
    adicionarItem(nome, preco) {  
        const novoItem = new ItemPedido(nome, preco); // [5]  
        this.itens.push(novoItem);  
    }  
}
```

```
// =====
```

// 2. CLASSE DE SERVIÇO (Regras de Negócio do Back-End)

```
// =====
```

```
class PedidoService {  
    constructor() {  
        this.bancoDeDadosSimulado = []; // Simula persistência no Back-End [1]  
    }  
}
```

```

// Método do serviço que coordena as regras de criação de um pedido
processarNovoPedido(id, clienteld, itensCarinho) {
    const pedido = new Pedido(id, clienteld);

    // Adiciona os itens recebidos pela requisição
    for (const item of itensCarinho) {
        // Regra de validação: impedir preços inválidos
        if (item.preco <= 0) {
            throw new Error("O preço do produto precisa ser maior que zero!");
        }
        pedido.adicionarItem(item.nome, item.preco);
    }

    // Persiste a entidade no banco de dados [1]
    this.bancoDeDadosSimulado.push(pedido);

    console.log(` [Back-End] Pedido #${pedido.id} salvo com sucesso para o Cliente #${pedido.clienteld}!` );
    return pedido;
}

// =====

// 3. EXECUÇÃO SIMULANDO REQUISIÇÃO WEB

// =====

const service = new PedidoService(); // [1]

// Dados que viriam de uma requisição Front-End (JSON) [1, 7]
const dadosRequisicao = {

```

```
pedidold: 101,  
clienteld: 442,  
itens: [  
  { nome: "Teclado Mecânico", preco: 250 },  
  { nome: "Mouse Gamer", preco: 150 }  
]  
};
```

```
// O serviço do Back-End recebe a requisição, valida, constrói e salva as entidades  
service.processarNovoPedido(dadosRequisicao.pedidold,  
dadosRequisicao.clienteld, dadosRequisicao.itens);
```

No JavaScript, a palavra-chave **this se refere ao objeto atual** no qual o código está sendo escrito ou executado. Pense nele como um pronome (como "este" ou "meu") que o objeto usa para se referir a si mesmo

Os Componentes Principais de uma Classe

Para construir e explicar o código do seu grupo, vocês precisam dominar os seguintes componentes de uma classe de acordo com o material técnico:

1. Declaração de Classe

Uma classe é definida usando a palavra-chave **class** seguida pelo nome do modelo (que, por convenção, começa com letra maiúscula).

- **Regra de Hoisting:** Ao contrário das funções normais em JavaScript, as declarações de classe **não sofrem *hoisting*** (não são "içadas" para o topo do código). Isso significa que você deve obrigatoriamente declarar a sua classe antes de tentar criar uma instância dela com o operador **new**.

2. O Construtor (constructor)

O construtor é um método especial usado para **criar e inicializar** as propriedades de um objeto.

- Só pode existir **um único método com o nome *constructor*** dentro de uma classe; caso contrário, o JavaScript disparará um erro de sintaxe (**SyntaxError**).

- Ele é responsável por criar o novo objeto, vincular a palavra-chave `this` a ele e executar as lógicas de inicialização.

3. Atributos e Campos Privados

Atributos são as propriedades ou dados armazenados por cada instância.

- **Campos Privados (#):** Se você declarar um atributo iniciando com o caractere `#` (como `#year` ou `#saldo`), o JavaScript impedirá que ele seja lido ou modificado fora do escopo da própria classe. Isso é a base do **encapsulamento**, protegendo o estado interno do objeto contra manipulações externas indevidas.

4. Métodos

São as funções definidas dentro do corpo da classe (após o construtor) que representam os comportamentos ou ações que o objeto pode realizar.

5. Herança (extends e super)

Para criar uma classe mais específica a partir de uma classe genérica (relação do tipo "é um"), usamos a palavra-chave `extends`.

- Se a subclasse tiver seu próprio construtor, ela **deve chamar o método `super()` antes de usar o `this`**. O `super()` invoca o construtor da classe pai para inicializar as propriedades herdadas.

6. Métodos Estáticos (static)

Definidos com a palavra-chave `static`, os métodos estáticos pertencem à classe em si e não às instâncias de objetos. Eles são chamados diretamente a partir do nome da classe (ex: `NomeDaClasse.metodoEstatico()`) e geralmente servem para criar funções utilitárias.

Conexão Prática: Classes no Back-End (Grupo 4)

No desenvolvimento Back-End, as classes são fundamentais para **organizar a arquitetura do servidor**, permitindo separar de forma limpa o que é **dado** do que é **comportamento**.

1. **Entidades de Domínio:** São as classes estruturadas que representam os dados de negócio do sistema (ex: as classes `Cliente`, `Pedido` e `ItemPedido`). Elas simulam o fluxo do mundo real, controlando como os dados se relacionam (associação, agregação e composição).
2. **Classes de Serviço:** São classes funcionais que não guardam estado, mas contêm as regras de negócio de um servidor Back-End. Elas processam os

dados que chegam por requisições e coordenam o salvamento no banco de dados.

Sim, vocês já fizeram tudo isso! Com as interações anteriores, o seu grupo concluiu com sucesso todas as etapas exigidas para a **Aula 2 (Aprofundamento)** do cronograma do seminário.

Vejam como o trabalho de vocês já cobre perfeitamente o **produto esperado** (que é o **Resumo técnico + código planejado**):

1. Seleção de Definições, Exemplos e Conexões (Resumo Técnico)

- **Associação, Agregação e Composição:** Vocês definiram claramente cada um dos três relacionamentos, utilizando analogias práticas.
- **A principal diferença:** Explicaram que a diferença crucial reside na **independência do ciclo de vida das partes** (se as partes podem ou não existir sem o objeto "todo").
- **Conexão Cliente, Pedido e ItemPedido:** Demonstraram como essas três classes interagem no mundo real, mapeando Cliente/Pedido como uma associação independente e Pedido/ItemPedido como uma composição dependente.
- **POO no Back-End (Entidades e Serviços):** Explicaram o papel de representar conceitos do mundo real por meio de **Entidades do Domínio** (dados) e coordenar as regras por meio de **Classes de Serviço** (ações/regras de negócio)